

# 基于 lammps-mace 程序包的熔盐分子模拟

从熔盐体系建模到 MACE 模型微调和 LAMMPS 分子模拟

Wenshuo Liang

2026-06-03

本文档面向已经具备 Python、Packmol、LAMMPS 和 MACE 基础运行环境的用户，说明如何使用 lammps-mace 包完成高通量熔盐体系的构盒、零样本 MACE-MD 输入生成、MACE 微调任务准备，以及微调模型 MD 输入生成。本文档是打印版教程，命令可直接复制到终端后按实际路径调整。

## Contents

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b> workflows overview</b>                         | <b>2</b>  |
| 1.1      | core stages                                        | 2         |
| 1.2      | directory convention                               | 2         |
| <b>2</b> | <b> installation and environment check</b>         | <b>2</b>  |
| 2.1      | install dependencies and start environment         | 2         |
| 2.2      | install Python package                             | 3         |
| 2.3      | run before check                                   | 4         |
| <b>3</b> | <b> prepare project skeleton</b>                   | <b>4</b>  |
| 3.1      | project directory                                  | 4         |
| 3.2      | CSV input format                                   | 5         |
| <b>4</b> | <b> batch construction of simulation boxes</b>     | <b>5</b>  |
| 4.1      | execute box construction                           | 5         |
| 4.2      | box construction output                            | 5         |
| <b>5</b> | <b> generate zero-shot MACE-MD input</b>           | <b>6</b>  |
| 5.1      | prepare base model                                 | 6         |
| 5.2      | generate and submit LAMMPS task                    | 6         |
| 5.3      | single task debug LAMMPS                           | 7         |
| <b>6</b> | <b> prepare MACE fine-tuning task</b>              | <b>8</b>  |
| <b>7</b> | <b> use fine-tuning model to generate MD input</b> | <b>9</b>  |
| <b>8</b> | <b> command quick reference</b>                    | <b>10</b> |
| 8.1      | lammps-mace init                                   | 11        |
| 8.2      | lammps-mace build-boxes                            | 11        |
| 8.3      | lammps-mace mace-md-npt                            | 11        |
| 8.4      | lammps-mace mace-md-nvt                            | 12        |
| 8.5      | lammps-mace finetune                               | 12        |

## 1 workflows 概览

lammps-mace 是一个围绕 MACE-MP、LAMMPS ML-IAP 和熔盐体系建模整理的命令行包。它的目标不是替代 LAMMPS、MACE 或 DFT 软件，而是把重复性强、容易写错路径的步骤收敛到一个稳定的 CLI。

### 1.1 核心阶段

推荐把一次完整流程拆成下面五个阶段：

1. 准备工作目录和 CSV 体系表。
2. 用 Packmol 和 ASE 从 CSV 批量构建 simulation\_box/box.data。
3. 基于基础 MACE-MP ML-IAP 模型生成零样本 LAMMPS 输入。
4. 用 extxyz 数据准备 MACE 微调任务目录和训练脚本。
5. 将微调后的 .model 转换为 ML-IAP .pt，再生成微调模型的 LAMMPS 输入。

### 1.2 目录约定

软件安装、源码和任务数据建议分开管理。远端机器上推荐如下布局：

```
/root/base/envs/mace-lammps-torch291-cu128
/root/base/src/lammps-mace
/root/base/src/lammps-develop
/root/base/build/lammps-mliap-torch291-cu128
/root/base/software/lammps-mliap-torch291-cu128
/root/shared-nvme/mace_lammps_smoke/outputs
```

/root/base 适合放 conda/venv、源码、编译目录和软件前缀。/root/shared-nvme 是共享卷，适合放任务、数据、模型和输出，不建议作为软件安装前缀。

## 2 安装与环境检查

### 2.1 安装依赖并启动环境

远端环境使用 conda prefix。应先加载 conda shell hook，再激活目标 prefix：

```
source /base/mambaforge/etc/profile.d/conda.sh
conda activate /root/base/envs/mace-lammps-torch291-cu128
```

如果环境还不存在，先创建并安装构盒阶段的基础依赖：

```
mkdir -p /root/base/envs
/base/mambaforge/bin/conda create -y \
  -p /root/base/envs/mace-lammps-torch291-cu128 \
  -c conda-forge \
  python=3.12 cmake ninja openmpi packmol

source /base/mambaforge/etc/profile.d/conda.sh
conda activate /root/base/envs/mace-lammps-torch291-cu128
```

随后安装 Python 侧依赖。普通 PyPI 包和 PyTorch/CUDA wheel 建议分开装：

```
PY=/root/base/envs/mace-lammps-torch291-cu128/bin/python

$PY -m pip install \
  -i https://mirrors.aliyun.com/pypi/simple/ \
  --trusted-host mirrors.aliyun.com \
  --extra-index-url https://pypi.tuna.tsinghua.edu.cn/simple \
  filelock typing-extensions sympy networkx jinja2 fsspec pillow \
  numpy==1.26.4 mpmath MarkupSafe

$PY -m pip install --no-deps \
  --index-url https://mirror.sjtu.edu.cn/pytorch-wheels/cu128 \
  torch==2.9.1+cu128 torchvision==0.24.1+cu128 torchaudio==2.9.1+cu128

$PY -m pip install --no-deps \
  --index-url https://mirror.sjtu.edu.cn/pytorch-wheels/cu128 \
  nvidia-cuda-nvrtc-cu12==12.8.93 \
  nvidia-cuda-runtime-cu12==12.8.90 \
  nvidia-cuda-cupti-cu12==12.8.90 \
  nvidia-cudnn-cu12==9.10.2.21 \
  nvidia-cublas-cu12==12.8.4.1 \
  nvidia-cufft-cu12==11.3.3.83 \
  nvidia-curand-cu12==10.3.9.90 \
  nvidia-cusolver-cu12==11.7.3.90 \
  nvidia-cuspars-cu12==12.5.8.93 \
  nvidia-cusparselt-cu12==0.7.1 \
  nvidia-nccl-cu12==2.27.5 \
  nvidia-nvshmem-cu12==3.3.20 \
  nvidia-nvtx-cu12==12.8.90 \
  nvidia-nvjitlink-cu12==12.8.93 \
  nvidia-cufile-cu12==1.13.1.3 \
  triton==3.5.1

$PY -m pip install \
  -i https://mirrors.aliyun.com/pypi/simple/ \
  --trusted-host mirrors.aliyun.com \
  --extra-index-url https://pypi.tuna.tsinghua.edu.cn/simple \
  numpy==1.26.4 "matplotlib<1.2" ase==3.28.0 mace-torch==0.3.16 \
  cupy-cuda12x==13.6.0 \
  cuequivariance==0.10.0 cuequivariance-torch==0.10.0 \
  cuequivariance-ops-torch-cu12==0.10.0
```

LAMMPS ML-IAP 需要单独源码编译。完成 LAMMPS 安装后，再把 LAMMPS 前缀和 Python 环境加入运行环境：

```
export MACE_ENV=/root/base/envs/mace-lammps-torch291-cu128
export LAMMPS_PREFIX=/root/base/software/lammps-ml-iap-torch291-cu128
export PATH=$LAMMPS_PREFIX/bin:$MACE_ENV/bin:$PATH
export LD_LIBRARY_PATH=$LAMMPS_PREFIX/lib:$MACE_ENV/lib:$LD_LIBRARY_PATH
export PYTHONPATH=$MACE_ENV/lib/python3.12/site-packages
```

## 2.2 安装 Python 包

在开发目录中安装当前包：

```
cd /root/base/src/lammps-mace
python -m pip install -e .
lammps-mace --help
```

从 GitHub 安装时可使用：

```
python -m pip install git+https://github.com/LiangWenshuo1118/lammps-mace.git
```

项目仓库地址：<https://github.com/LiangWenshuo1118/lammps-mace>。

安装后提供的命令名是 lammps-mace，Python import 包名是 lammps\_mace。

## 2.3 运行前检查

建模阶段依赖 ASE 和 Packmol；MD 和微调阶段依赖 MACE、PyTorch、LAMMPS ML-IAP 和已转换好的模型文件。正式运行前建议先检查：

```
python -c "import ase; print(ase.__version__)"
packmol -h
lammps-mace --version
```

如果需要在 LAMMPS 中运行 MACE ML-IAP，还应确认：

```
LD_LIBRARY_PATH=/path/to/lammps-ml-iap-install/lib:$LD_LIBRARY_PATH \
/path/to/lammps-ml-iap-install/bin/lmp -h

python -c "import lammps; L=lammps.lammps(cmdargs=['-log','none','-screen','none']); print(L.version()); L.close()"
```

## 3 准备项目骨架

新建一个演示目录：

```
lammps-mace init --target demo
cd demo
```

这个命令会创建项目目录，并写出一个示例 CSV。

### 3.1 项目目录

初始化后的目录结构如下：

```
demo/
├── examples/systems.csv
├── outputs/
├── models/base/
├── models/finetuned/
├── data/finetune/
└── data/dft/
```

- `examples/systems.csv` 是示例体系表，也是构盒阶段最小输入文件。
- `outputs/` 用于存放构盒结果、LAMMPS 输入、运行脚本、轨迹、日志和训练任务输出。
- `models/base/` 用于存放基础 MACE 模型，例如 `mace-mp-0b3-medium.model` 及其 ML-IAP 转换文件。
- `models/finetuned/` 用于集中存放希望长期保留或复用的微调模型。
- `data/dft/` 用于存放 DFT 任务数据、原始计算结果或后续数据处理所需的中间文件。

- `data/finetune/` 用于存放从 DFT 数据中提取并整理好的 MACE 微调数据，例如训练集 `Training.extxyz` 和验证集 `Validation.extxyz`。

### 3.2 CSV 输入格式

CSV 是唯一的初始体系输入文件。`system.json` 和 `md_settings.json` 两个文件是 workflow 生成的中间元数据。

| 字段                              | 示例            | 说明                                                |
|---------------------------------|---------------|---------------------------------------------------|
| <code>system_index</code>       | 001           | 体系编号，用于输出目录命名。                                    |
| <code>salts</code>              | LiCl-KCl-AIF3 | 盐公式，多个盐用短横线、逗号或分号分隔。                              |
| <code>concentrations</code>     | 50-45-5       | 与 <code>salts</code> 一一对应的浓度。                     |
| <code>concentration_type</code> | mol%          | 支持 mol% 或 wt%。                                    |
| <code>total_atoms</code>        | 128           | 目标原子数。实际原子数会因整数 <code>formula unit</code> 近似略有调整。 |

## 4 批量构建模拟盒子

### 4.1 执行构盒

从 CSV 构建所有体系：

```
lammps-mace build-boxes \
  --csv examples/systems.csv \
  --output-root outputs \
  --density 1.46
```

`--packmol-tolerance` 为可选参数，默认值为 **2.0**。

命令会读取每一行体系定义，计算盐的 `formula unit` 数、元素原子数和盒长，然后调用 Packmol 随机填充盒子，最后用 ASE 转出 LAMMPS data 文件。

### 4.2 构盒输出

每一行 CSV 对应一个输出目录，例如：

```
outputs/001_LiCl/
├── simulation_box/
│   ├── ions/
│   │   ├── Li.pdb
│   │   └── Cl.pdb
│   ├── packmol.inp
│   ├── packmol.log
│   ├── box.pdb
│   ├── box.data
│   └── system.json
```

其中：

- `simulation_box/box.data` 是后续 LAMMPS 输入读取的结构文件。
- `simulation_box/system.json` 记录归一化后的组成、密度、元素计数和盒长。
- `simulation_box/packmol.log` 用来排查 Packmol 是否成功收敛。

## 5 生成零样本 MACE-MD 输入

### 5.1 准备基础模型

基础 MACE-MP checkpoint 建议放在：

```
models/base/mace-mp-0b3-medium.model
```

LAMMPS ML-IAP 不能直接读取 `.model`，需要先转换为 `*-mliap_lammps.pt`：

```
python -m mace.cli.create_lammps_model \  
models/base/mace-mp-0b3-medium.model \  
--format=mliap
```

转换后通常得到：

```
models/base/mace-mp-0b3-medium.model-mliap_lammps.pt
```

### 5.2 生成并提交 LAMMPS 任务

第一步先生成 NPT 任务。NPT 用来在指定温度和压力下弛豫体系：

```
lammps-mace mace-md-npt \  
--boxes-root outputs \  
--model-path models/base/mace-mp-0b3-medium.model-mliap_lammps.pt \  
--temperature 973 1073 \  
--pressure 0 1 \  
--npt-steps 1000 \  
--timestep-fs 1.0 \  
--thermo-every 100 \  
--dump-every 100
```

`--temperature` 和 `--pressure` 都支持多个值。NPT 会为二者的笛卡尔积创建条件目录，例如 `973K_00bar`、`973K_01bar` 和 `1073K_00bar`。

NPT 任务输出结构如下：

```
outputs/001_LiCl/  
├─ simulation_box/  
│   └─ box.data  
├─ 973K_00bar/  
│   └─ mace-md-npt/  
│       ├── in.lammps  
│       ├── md_settings.json  
│       └─ run.sh
```

同时会在 `outputs/` 下生成 NPT 总脚本：

```
outputs/run_mace_md_npt_all.sh
```

提交全部 NPT 任务：

```
bash outputs/run_mace_md_npt_all.sh
```

NPT 总脚本会按体系目录和条件目录顺序调用每个 `mace-md-npt/run.sh`。任务成功结束后，会在对应目录写入空文件 `task_finished`；如果任务中断，重新执行总脚本时，已有 `task_finished` 的目录会自动跳过。

第二步再生成 NVT 任务。NVT 会跟随已有的 NPT 条件目录生成输入。默认 NVT 读取初始构盒结构，因此不依赖 NPT 是否已经完成：

```
lammps-mace mace-md-nvt \
  --boxes-root outputs \
  --model-path models/base/mace-mp-0b3-medium.model-mliap_lammps.pt \
  --nvt-steps 1000 \
  --timestep-fs 1.0 \
  --thermo-every 100 \
  --dump-every 100
```

如果只想为部分温度生成 NVT，可在 NVT 命令中加入 `--temperature`，后面填写需要保留的温度值，例如只生成 973 K 条件对应的 NVT 输入。

如果希望 NVT 盒子使用 NPT 后段平均密度，可在 NVT 命令中加入 `--use-npt-density`，后面填写用于平均的 NPT 后段比例，例如 0.8 表示使用最后 80% 的密度样本。

使用 `--use-npt-density` 时，需要对应条件目录下的 NPT 已经运行完成，并存在下面两个文件：

```
mace-md-npt/final_mace_npt.data
mace-md-npt/log.kk.half.lammps
```

NVT 任务输出结构如下：

```
outputs/001_LiCl/
├── simulation_box/
│   └── box.data
└── 973K_00bar/
    └── mace-md-nvt/
        ├── in.lammps
        ├── md_settings.json
        └── run.sh
```

同时会在 `outputs/` 下生成 NVT 总脚本：

```
outputs/run_mace_md_nvt_all.sh
```

提交全部 NVT 任务：

```
bash outputs/run_mace_md_nvt_all.sh
```

NVT 总脚本会按已有 NPT 条件目录顺序调用每个 `mace-md-nvt/run.sh`。NVT 任务同样使用 `task_finished` 记录完成状态，重新提交时会自动跳过已完成目录。可用下面命令统一监控 NPT 和 NVT 进度：

```
find outputs -name task_finished -print | sort
```

### 5.3 单任务调试 LAMMPS

远端实测推荐使用上一节生成的总脚本顺序执行全部任务。单任务调试时，也可手动使用 KOKKOS ML-IAP 路径，并显式启用 half neighbor list：

```
export MACE_ENV=/root/base/envs/mace-lammps-torch291-cu128
export LAMMPS_PREFIX=/root/base/software/lammps-mliap-torch291-cu128
export PATH=$LAMMPS_PREFIX/bin:$MACE_ENV/bin:$PATH
export LD_LIBRARY_PATH=$LAMMPS_PREFIX/lib:$MACE_ENV/lib:$LD_LIBRARY_PATH
export PYTHONPATH=$MACE_ENV/lib/python3.12/site-packages
export OMPI_MCA_mtl=^ofi
export OMPI_MCA_pml=obl
```

```
export TORCH_FORCE_NO_WEIGHTS_ONLY_LOAD=1

cd outputs/001_LiCl/973K_00bar/mace-md-nvt
$LAMMPS_PREFIX/bin/lmp \
  -k on g 1 -sf kk -pk kokkos neigh half \
  -in in.lammps -log log.kk.half.lammps
```

日志中出现 Loop time 通常表示 smoke 运行完成。

## 6 准备 MACE 微调任务

MACE 官方文档推荐，正式迁移 foundation model 时优先考虑 multihead replay fine-tuning，以降低遗忘风险；这需要额外准备 replay/pretraining 数据，例如 `--pt_train_file`。本教程先采用官方 naive fine-tuning 示例作为通用起点：关闭 multihead、使用 `E0s=average`、较小学习率、EMA、AMSGrad 和 float64。参考：[MACE fine-tuning 文档](#)。

本章示例使用的微调数据来自 SuperSalt，放在 `data/finetune/` 目录下，训练集和验证集分别命名为 `Training.extxyz` 和 `Validation.extxyz`。如果后续替换为自己的 DFT 数据，只需要保持相同的 `extxyz` 文件格式，并在命令中调整文件名即可。

使用下面这一组命令准备微调任务目录：

```
lammps-mace finetune \
  --task-dir outputs/finetune-mace \
  --dataset-root data/finetune \
  --train-file Training.extxyz \
  --valid-file Validation.extxyz \
  --foundation-model models/base/mace-mp-0b3-medium.model \
  --name mace-mp-0b3-medium-finetuned \
  --max-num-epochs 6 \
  --batch-size 2 \
  --forces-key force \
  --stress-key stress \
  --stress-weight 1.0 \
  --e0s average \
  --energy-weight 1.0 \
  --forces-weight 1.0 \
  --lr 0.01 \
  --scaling rms_forces_scaling \
  --ema \
  --ema-decay 0.99 \
  --amsgrad \
  --default-dtype float64 \
  --seed 3 \
  --no-multiheads-finetuning \
  --smoke-max-configs 100 \
  --require-dataset
```

`--smoke-max-configs` 是可选参数，只用于 smoke 测试。设置后，程序会在任务目录内写出裁剪后的 `smoke-dataset/`，训练脚本会指向这个小数据集，用来快速检查训练链路。正式训练时可以删除这个参数。

输出目录：

```
outputs/finetune-mace/
├─ checkpoints/
├─ logs/
├─ models/
├─ results/
└─ finetune_settings.json
```



```
└─ run_train.sh
```

run\_train.sh 中真正执行的是 MACE 的 mace\_run\_train 命令。其中核心内容类似下面这样：

```
cd "$(dirname "$0")"
exec mace_run_train \
  --name mace-mp-0b3-medium-finetuned \
  --foundation_model /root/demo/models/base/mace-mp-0b3-medium.model \
  --train_file /root/demo/outputs/finetune-mace/smoke-dataset/Training.
  extxyz \
  --valid_file /root/demo/outputs/finetune-mace/smoke-dataset/Validation.
  extxyz \
  --results_dir /root/demo/outputs/finetune-mace/results \
  --work_dir /root/demo/outputs/finetune-mace \
  --log_dir /root/demo/outputs/finetune-mace/logs \
  --model_dir /root/demo/outputs/finetune-mace/models \
  --checkpoints_dir /root/demo/outputs/finetune-mace/checkpoints \
  --E0s average \
  --valid_batch_size 2 \
  --device cuda \
  --max_num_epochs 6 \
  --batch_size 2 \
  --energy_key energy \
  --forces_key force \
  --stress_key stress \
  --energy_weight 1.0 \
  --forces_weight 1.0 \
  --stress_weight 1.0 \
  --lr 0.01 \
  --scaling rms_forces_scaling \
  --default_dtype float64 \
  --seed 3 \
  --multiheads_finetuning False \
  --ema \
  --ema_decay 0.99 \
  --amsgrad
```

提交训练任务：

```
cd outputs/finetune-mace
nohup bash run_train.sh > nohup.out 2>&1 &
```

## 7 使用微调模型生成 MD 输入

第 6 章提交训练任务时已经进入 outputs/finetune-mace 目录。训练完成后，先回到项目根目录：

```
cd ../../
```

第 6 章示例中 --name 设置为 mace-mp-0b3-medium-finetuned。训练完成后，模型通常位于 outputs/finetune-mace/models/mace-mp-0b3-medium-finetuned.model。先把这个 .model 转成 ML-IAP 格式：

```
python -m mace.cli.create_lammps_model \
  outputs/finetune-mace/models/mace-mp-0b3-medium-finetuned.model \
  --format=mliap
```

需要注意的是，按照本文的微调流程导出的模型不一定继续保留基础 MACE-MP 的全部元素覆盖范围。微调后的模型通常会以微调数据覆盖的元素表为准，因此模型文件可能明显变小，但适用体系也会变窄。SuperSalt 数据集为例，微调模型只支持 Li, Na, Mg, Cl, K, Ca, Zn, Rb, Sr, Zr, Cs, Ba。如果后续体系包含微调模型不支持的元素，例如 Al 或 F，LAMMPS 会在 pair\_coeff 阶段报错。实际使用前应确认 simulation\_box/system.json 里的元素都在微调模型支持范围内。

再生成微调模型输入。命令仍然使用 NPT/NVT 两个子命令，只需要把--model-path 指向新的 ML-IAP 模型。为避免覆盖第 5 章基础模型的 mace-md-npt 和 mace-md-nvt 目录，这里加上--task-suffix finetuned。先生成并提交微调模型 NPT 任务：

```
lammps-mace mace-md-npt \  
  --boxes-root outputs \  
  --model-path outputs/finetune-mace/models/mace-mp-0b3-medium-finetuned.  
    model-mliap_lammps.pt \  
  --task-suffix finetuned \  
  --temperature 973 \  
  --pressure 0 \  
  --npt-steps 100 \  
  --timestep-fs 1.0 \  
  --thermo-every 10  
  
bash outputs/run_mace_md_npt_finetuned_all.sh
```

等待 NPT 完成后，再生成并提交 NVT 任务。这里使用 --use-npt-density 0.8，表示用 NPT 最后 80% 的密度平均值确定 NVT 盒子尺寸：

```
lammps-mace mace-md-nvt \  
  --boxes-root outputs \  
  --model-path outputs/finetune-mace/models/mace-mp-0b3-medium-finetuned.  
    model-mliap_lammps.pt \  
  --task-suffix finetuned \  
  --nvt-steps 100 \  
  --timestep-fs 1.0 \  
  --thermo-every 10 \  
  --use-npt-density 0.8  
  
bash outputs/run_mace_md_nvt_finetuned_all.sh
```

输出结构与基础模型相同：

```
outputs/001_LiCl/973K_00bar/  
├─ mace-md-npt-finetuned/in.lammps  
└─ mace-md-nvt-finetuned/in.lammps
```

运行方式与基础模型 MACE-MD 相同。总脚本为：

```
outputs/run_mace_md_npt_finetuned_all.sh  
outputs/run_mace_md_nvt_finetuned_all.sh
```

## 8 命令速查

本章按 lammps-mace 的二级命令整理常用写法。每个命令默认从项目根目录执行，除非正文特别说明已经切换到某个任务目录。

| 一级子命令                    | 功能                                                                                     |
|--------------------------|----------------------------------------------------------------------------------------|
| <code>init</code>        | 创建项目骨架、示例 CSV 和常用目录。                                                                   |
| <code>build-boxes</code> | 根据 CSV 批量构建模拟盒子，并把 Packmol 结果写入各体系的 <code>simulation_box/</code> 。                     |
| <code>mace-md-npt</code> | 根据已有盒子和 ML-IAP 模型生成 NPT LAMMPS 输入、单任务运行脚本和总提交脚本。                                       |
| <code>mace-md-nvt</code> | 根据已有 NPT 条件目录生成 NVT 输入；可选择继续使用初始盒子，或用 NPT 后段平均密度确定盒长。                                  |
| <code>finetune</code>    | 创建 MACE 微调任务目录，写出 <code>run_train.sh</code> 和精简的 <code>finetune_settings.json</code> 。 |

## 8.1 lammps-mace init

| 二级子命令               | 类型               | 功能                                                                                                                                                             |
|---------------------|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>target</code> | <code>str</code> | 必须。目标目录；在其中初始化项目骨架，写入 <code>examples/systems.csv</code> ，并创建 <code>models/</code> 、 <code>data/</code> 和 <code>outputs/</code> 等常用目录。该命令不构盒、不下载模型，也不启动 LAMMPS。 |

## 8.2 lammps-mace build-boxes

| 二级子命令                          | 类型                 | 功能                                              |
|--------------------------------|--------------------|-------------------------------------------------|
| <code>csv</code>               | <code>str</code>   | 必须。体系 CSV；CSV 中的组成用于构盒，MD 温度由后续 NPT/NVT 命令指定。   |
| <code>output-root</code>       | <code>str</code>   | 可选。体系输出根目录；默认 <code>outputs/</code> 。           |
| <code>density</code>           | <code>float</code> | 必须。初始建模密度，单位为 <code>g/cm3</code> 。              |
| <code>packmol-tolerance</code> | <code>float</code> | 可选。Packmol 最小原子间距；默认 <code>2.0</code> 。         |
| <code>packmol-bin</code>       | <code>str</code>   | 可选。Packmol 可执行文件名称或路径；默认 <code>packmol</code> 。 |
| <code>seed</code>              | <code>int</code>   | 可选。Packmol 随机种子，用于复现初始构型；默认 <code>973</code> 。  |

## 8.3 lammps-mace mace-md-npt

| 二级子命令                    | 类型                       | 功能                                                                                          |
|--------------------------|--------------------------|---------------------------------------------------------------------------------------------|
| <code>boxes-root</code>  | <code>str</code>         | 可选。已完成构盒的体系根目录；默认 <code>outputs/</code> ，该目录下应包含各体系的 <code>simulation_box/</code> 。         |
| <code>model-path</code>  | <code>str</code>         | 可选。已转换好的 ML-IAP 模型文件；默认 <code>models/base/mace-mp-0b3-medium.model-mliap_lammps.pt</code> 。 |
| <code>temperature</code> | <code>list[float]</code> | 必须。NPT 温度；支持一次给多个温度。                                                                        |
| <code>pressure</code>    | <code>list[float]</code> | 必须。NPT 压力；支持一次给多个压力。                                                                        |

| 二级子命令        | 类型    | 功能                                                                                      |
|--------------|-------|-----------------------------------------------------------------------------------------|
| task-suffix  | str   | 可选。任务目录和总脚本后缀；默认无后缀，例如 <code>finetuned</code> 会生成 <code>mace-md-npt-finetuned/</code> 。 |
| npt-steps    | int   | 可选。NPT 步数；默认 <b>1000</b> 。                                                              |
| timestep-fs  | float | 可选。LAMMPS 时间步，单位为 fs；默认 <b>1.0</b> 。                                                    |
| thermo-every | int   | 可选。thermo 输出间隔；默认 <b>100</b> 。                                                          |
| dump-every   | int   | 可选。轨迹 dump 输出间隔；默认 <b>100</b> 。                                                         |
| tdamp-fs     | float | 可选。温控阻尼时间，单位为 fs；默认 <b>100.0</b> 。                                                      |
| pdamp-fs     | float | 可选。压控阻尼时间，单位为 fs；默认 <b>1000.0</b> 。                                                     |

## 8.4 lammps-mace mace-md-nvt

| 二级子命令           | 类型          | 功能                                                                                                                         |
|-----------------|-------------|----------------------------------------------------------------------------------------------------------------------------|
| boxes-root      | str         | 可选。体系根目录；默认 <code>outputs/</code> ，程序从已有 NPT 条件目录判断需要生成哪些 NVT 任务。                                                          |
| model-path      | str         | 可选。NVT 使用的 ML-IAP 模型；默认 <code>models/base/mace-mp-0b3-medium.model-mliap_lammps.pt</code> 。使用微调模型时必须显式指定。                  |
| temperature     | list[float] | 可选。温度过滤；默认处理所有已有 NPT 条件，设置后只处理指定温度。                                                                                        |
| use-npt-density | float       | 可选。NVT 密度来源；默认使用 <code>simulation_box/box.data</code> ，例如 <b>0.8</b> 表示使用 NPT 最后 80% 的密度平均值确定 NVT 盒长。                      |
| task-suffix     | str         | 可选。任务后缀；默认无后缀，例如 <code>finetuned</code> 只查找 <code>mace-md-npt-finetuned/</code> ，并生成 <code>mace-md-nvt-finetuned/</code> 。 |
| nvt-steps       | int         | 可选。NVT 步数；默认 <b>1000</b> 。                                                                                                 |
| timestep-fs     | float       | 可选。LAMMPS 时间步，单位为 fs；默认 <b>1.0</b> 。                                                                                       |
| thermo-every    | int         | 可选。thermo 输出间隔；默认 <b>100</b> 。                                                                                             |
| dump-every      | int         | 可选。轨迹 dump 输出间隔；默认 <b>100</b> 。                                                                                            |
| tdamp-fs        | float       | 可选。温控阻尼时间，单位为 fs；默认 <b>100.0</b> 。                                                                                         |

## 8.5 lammps-mace finetune

| 二级子命令        | 类型  | 功能                                                                                                                                          |
|--------------|-----|---------------------------------------------------------------------------------------------------------------------------------------------|
| task-dir     | str | 可选。微调任务目录；默认 <code>outputs/finetune-mace/</code> ，程序会在其中写入 <code>run_train.sh</code> 、 <code>finetune_settings.json</code> 、日志目录、模型目录和结果目录。 |
| dataset-root | str | 可选。extxyz 数据目录；默认 <code>data/finetune/</code> ，本教程示例数据来自 SuperSalt。                                                                         |
| train-file   | str | 可选。训练集 extxyz 文件名；默认 <code>train.extxyz</code> 。                                                                                            |

| 二级子命令                 | 类型    | 功能                                                              |
|-----------------------|-------|-----------------------------------------------------------------|
| valid-file            | str   | 可选。验证集 extxyz 文件名；默认 valid.extxyz。                              |
| test-file             | str   | 可选。测试集 extxyz 文件名；默认不使用测试集。                                     |
| foundation-model      | str   | 可选。未转换的 MACE .model 文件；默认 models/base/mace-mp-0b3-medium.model。 |
| name                  | str   | 可选。训练输出模型名称；默认 mace-mp-0b3-medium-finetuned。                    |
| results-dir           | str   | 可选。MACE 结果目录；默认 <task-dir>/results/。                            |
| work-dir              | str   | 可选。MACE 工作目录；默认 <task-dir>/。                                    |
| log-dir               | str   | 可选。训练日志目录；默认 <task-dir>/logs/。                                  |
| model-dir             | str   | 可选。训练输出模型目录；默认 <task-dir>/models/。                              |
| checkpoints-dir       | str   | 可选。checkpoint 目录；默认 <task-dir>/checkpoints/。                    |
| device                | str   | 可选。训练设备；默认 cuda，也可设为 cpu。                                       |
| max-num-epochs        | int   | 可选。最大训练 epoch 数；默认 6。                                           |
| batch-size            | int   | 可选。训练 batch size；默认 2。                                          |
| valid-batch-size      | int   | 可选。验证 batch size；默认跟 --batch-size 一致。                           |
| energy-key            | str   | 可选。extxyz 中能量字段名称；默认 energy。                                    |
| forces-key            | str   | 可选。extxyz 中力字段名称；默认 forces。                                     |
| stress-key            | str   | 可选。extxyz 中应力字段名称；默认不训练应力。                                      |
| e0s                   | str   | 可选。原子参考能设置；默认 average。                                          |
| energy-weight         | float | 可选。能量损失权重；默认 1.0。                                               |
| forces-weight         | float | 可选。力损失权重；默认 1.0。                                                |
| stress-weight         | float | 可选。应力损失权重；默认不传入。                                                |
| r-max                 | float | 可选。邻居截断半径；默认不传入，使用 mace_run_train 默认值。                          |
| lr                    | float | 可选。学习率；默认 0.01。                                                 |
| scaling               | str   | 可选。数据缩放方式；默认 rms_forces_scaling。                                |
| ema                   | bool  | 可选。是否启用 EMA；默认启用。                                               |
| ema-decay             | float | 可选。EMA decay；默认 0.99。                                           |
| amsgrad               | bool  | 可选。是否启用 AMSGrad；默认启用。                                           |
| default-dtype         | str   | 可选。默认浮点精度；默认 float64。                                           |
| seed                  | int   | 可选。随机种子；默认 3。                                                   |
| multiheads-finetuning | bool  | 可选。是否启用 multihead fine-tuning；默认关闭。                             |
| smoke-max-configs     | int   | 可选。只用于 smoke 测试，生成小数据集；默认不裁剪数据，正式训练时可以删除。                       |
| require-dataset       | bool  | 可选。要求训练/验证 extxyz 文件必须存在；默认不强制，缺文件时只打印 warning。                 |

教程结束。建议把本 PDF 与当前代码版本一起归档，方便复现实验环境。